

Universidad Nacional Autónoma de México

Facultad de Ciencias

Cómputo Concurrente

2027-1

Práctica 4. Spinlocks y algunas Primitivas

ALUMNOS:

Martínez Hidalgo Paola Mildred - 319300217
Dominguez Sanchez Gustavo Alonso - 418047121

Fecha de entrega:
08 de octubre de 2026

Profesor:
Gilde Valeria Rodríguez



Ejercicios

Considérese lo siguiente:

- Persona 1 (P_1): Mildred.
  - Sistema Operativo: Windows 11
  - Procesador: AMD Ryzen 3 3250U Radeon Graphics
  - Hilos (máx): 4
- Persona 2 (P_2): Alonso.
  - Sistema Operativo: Fedora Linux 42
  - Procesador: Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz
  - Hilos (máx): 8

1. Utiliza el programa `RunSpin` para probar cada uno de los spinlocks, cada integrante en el equipo debe completar la siguiente tabla: (*max-1* es el número máximo de hilos en tu computadora menos uno).

Cuadro 1: Tabla comparativa cuando la tarea es muy breve

SpinLocks / Counter	**TAS**	**TTAS**	**Backoff**	**MCS**	**ALock**	**CLHLock**	**Reentrant**	**Counter Atomic**
Tiempo de ejecución: 400 tareas con 4 hilos	P_1 : 22.0ms P_2 : 06.7ms	P_1 : 27.3ms P_2 : 05.8ms	P_1 : 39.3ms P_2 : 26.6ms	P_1 : 33.4ms P_2 : 07.9ms	P_1 : 29.0ms P_2 : 07.4ms	P_1 : 22.4ms P_2 : 08.5ms	P_1 : 22.1ms P_2 : 06.5ms	P_1 : 20.2ms P_2 : 08.8ms
Tiempo de ejecución: 1000 tareas con 4 hilos	P_1 : 26.4ms P_2 : 10.6ms	P_1 : 28.4ms P_2 : 08.9ms	P_1 : 44.5ms P_2 : 33.2ms	P_1 : 31.7ms P_2 : 11.5ms	P_1 : 29.1ms P_2 : 10.3ms	P_1 : 39.5ms P_2 : 12.9ms	P_1 : 17.2ms P_2 : 10.8ms	P_1 : 23.7ms P_2 : 09.9ms
Tiempo de ejecución: 1000 tareas con max-1 hilos	P_1 : 18.9ms P_2 : 11.4ms	P_1 : 18.6ms P_2 : 11.5ms	P_1 : 31.1ms P_2 : 36.8ms	P_1 : 26.8ms P_2 : 10.9ms	P_1 : 22.9ms P_2 : 24.5ms	P_1 : 32.0ms P_2 : 12.4ms	P_1 : 20.3ms P_2 : 12.6ms	P_1 : 19.1ms P_2 : 11.7ms

2. Modificación del método `task(lock)`. En lugar de incrementar un contador, modifica la función `task(lock)` para que los hilos trabajen con una matriz compartida de tamaño $n \times n$, donde $n = 10$. Cada hilo deberá asignar valores a la matriz dentro de una sección crítica protegida por `lock` y, una vez completada la asignación, imprimir su contenido. Implementa una función separada para la asignación y la impresión de la matriz. Cada integrante del equipo debe completar la siguiente tabla:
   Se implementó el programa `RunSpinMatriz` tomando como base `RunSpin`, modificandolo para cumplir con lo que se pide: se declara una matriz (que compartirán todos) con un método para llenarla de valores (`asignarMatriz()`) e imprimirla (`imprimirMatriz()`), esto para lograr una tarea "pesada" (al menos, más pesada que la de un contador). Se modifica entonces el método `task()` en que ahora la sección crítica será la ejecución de estos dos nuevos métodos de la matriz. Por lo anterior, se eliminan las declaraciones y/o uso del `CounterAtomic`, ya que no son necesarias.

Cuadro 2: Tabla comparativa cuando la tarea es más pesada

SpinLocks / Counter	**TAS**	**TTAS**	**Backoff**	**MCS**	**ALock**	**CLHLock**	**Reentrant**
Tiempo de ejecución: 100 tareas con 4 hilos	P_1 : 32457.4 P_2 : 358.2ms	P_1 : 21863.6 P_2 : 171.9ms	P_1 : 21457.3 P_2 : 120.1ms	P_1 : 22759.0 P_2 : 144.3ms	P_1 : 17845.9 P_2 : 153.3ms	P_1 : 12119.6 P_2 : 169.8ms	P_1 : 4142.3ms P_2 : 114.8ms
Tiempo de ejecución: 100 tareas con max-1 hilos	P_1 : 32024.9 P_2 : 249.2ms	P_1 : 24147.2 P_2 : 225.6ms	P_1 :21867.6 P_2 : 191.3ms	P_1 : 21457.1 P_2 : 294.5ms	P_1 : 19547.5 P_2 : 318.8ms	P_1 : 15012.0 P_2 : 351.8ms	P_1 : 5129.9ms P_2 : 109.3ms

3. Escojan al integrante del equipo con mayor número de hilos. ¿Cuáles son las 2 implementaciones más rápidas del inciso 1? ¿A qué crees se debe?

   Definiendo a P_2 como el integrante con más hilos (8), basaremos el análisis en sus resultados. Además tomaremos en cuenta principalmente la prueba de mayor contención (1000 tareas con 7 hilos). De esta forma las implementaciones más rápidas del ejercicio 1, fueron `MCSLock` con 10.9ms y `TASLock` con 11.4ms.

   Respecto a `MCSLock` su velocidad puede deberse a la forma en la que esta implementación maneja la contención. Este candado forma a los hilos en una cola usando nodos, pero es importante notar que a diferencia de otros candados, cada hilo en espera hace spin verificando únicamente la bandera `locked` de su propio nodo loca, en vez de una variable global. Esto evita la necesidad de revisar la memoria caché del procesador ahorrando tiempo valioso, permitiendo que la implementación se mantenga eficiente al aumentar los hilos.

   Por otro lado, tenemos que `TASLock`, el cual, debido a la naturaleza de su diseño que es intentar acceder continuamente a la memoria RAM con la primitiva `getAndSet(true)`; y además considerando que el costo tan bajo de la tarea en este ejercicio (un contador); la culminación de estas tareas (sección crítica) breves son tan rápidas que sale factible usar estas primitivas para evitar gastar más tiempo esperando. Por lo que para tareas con un mayor overhead o costo, esta estrategia se verá mermada porque al ocupar más tiempo en las tareas (como en el ejercicio 2), se gasta innecesariamente ciclos de CPU al intentar acceder a cada momento al candado.

4. Escojan al integrante con mayor número de hilos. ¿Cuáles son las 2 implementaciones más rápidas del inciso 2? ¿A qué crees se debe?

   Definiendo nuevamente a P_2 como el integrante con más hilos (8), tomando en cuenta principalmente la prueba de mayor contención (100 tareas con 7 hilos), las implementaciones más rápidas fueron `ReentrantLock` con 109.3ms y `BackoffLock` con 191.3ms.

   Respecto a `ReentrantLock`, su velocidad puede deberse a que, a diferencia de los spinlocks, los hilos que no logran obtener el candado no esperan activamente, sino que son suspendidos por el sistema operativo y no consumen procesador mientras esperan. Esto es especialmente importante en este ejercicio, ya que la sección crítica es mucho más larga (asignar e imprimir una matriz de 10×10), por lo que los hilos en espera tardarían bastante tiempo girando sin hacer trabajo útil. Al suspenderse, el procesador queda libre para el hilo que sí tiene el candado, y es posible que por eso su tiempo casi no cambió al pasar de 4 a 7 hilos (114.8ms y 109.3ms respectivamente).

   Por otro lado, `BackoffLock` sí espera activamente, pero cuando un hilo falla al intentar obtener el candado no reintenta de inmediato, sino que espera un tiempo aleatorio antes de volver a intentar. Esto reduce la cantidad de accesos a la variable compartida y además hay menos hilos compitiendo por el procesador con el hilo que tiene el candado. Dado que la tarea es larga, esperar un poco antes de reintentar casi no perjudica, porque el candado tardará en liberarse de todas formas.

5. ¿Cuáles son las implementaciones que satisfacen la propiedad de Justicia? (Por el momento ignora `ReentrantLock`). Argumenta cómo lo hacen.
   Recordemos brevemente que una implementación satisface la propiedad de Justicia siempre que garantiza un orden (primero que llega, primero que se ejecuta) para evitar la inanición. Esta propiedad, similar a la propiedad FIFO, es justamente lo que nos puede indicar que candados cumplen esta propiedad: los que se basan en colas. En particular, cumplen con Justicia los siguientes candados:

   - `ALock`: Este candado garantiza la propiedad mediante un sistema de turnos que son secuenciales en un arreglo (circular). Cuando un hilo intenta usar el candado, utiliza `tail.getAndIncrement()` para obtener un número de turno (`slot`) único y se calcula según su oren de llegada. El hilo gira activamente esperando solo en ese espacio específico del arreglo (`flag[mySlotIndex.get()]`), y al terminar, otorga el turno al siguiente hilo que sigue en el arreglo.
   - `CLHLock`: Utiliza una estructura de cola construida con referencias atómicas (`AtomicReference<QNode>`). La justicia se logra porque cada hilo que llega utiliza `tail.getAndSet(qnode)` para formarse (de manera atómica) al final de la cola. Al hacerlo, obtiene una referencia de su predecesor y gira esperando únicamente a que predecesor específico libere el candado (`while (pred.locked)`), respetando el orden de formación asignado.
   - `MCSLock`: Similar al anterior, forma a los hilos en una lista utilizando `queue.getAndSet(qnode)`. En este caso, la justicia está en que el hilo predecesor apunta directamente al hilo sucesor (`pred.next = qnode`). CUando el predecesor termina, este mismo es quien le avisa al hilo que siue en la fila que ya puede pasar (`qnode.next.locked = false`), asegurándose así, que el acceso a la sección crítica mantenga el orden asignado.

   En los demás hilos como `TASLock`, `TTASLock` o `BackoffLock` no se cumple esto. En estas implementaciones todos los hilos compiten al mismo tiempo por el control del candado, esto porque utilizan operaciones como `get()` o `getAndSet(true)` cada vez que el candado se libera. Como no hay un registro que guarde el orden de llegada, el acceso de un hilo al candado como sabemos, puede ser porqué es más rápido o por pura suerte, por lo que existe la posibilidad de que alguno se quede esperando indefinidamente (inanición).